

Guide Question

1. Why are dynamic arrays preferred over fixed-size static arrays for unpredictable batch workloads?

Dynamic arrays are preferred because their capacity can grow when more data is added. A fixed-size array can run out of space if the workload becomes larger than expected. However, resizing has a hidden cost because a new memory block must be allocated and the existing elements must be copied, which takes $O(n)$ time.

2. What is the role of the destructor in managing heap allocation safety?

The destructor automatically releases the memory allocated by the class using `"delete[]"`. If memory is not released in a long-running application, memory leaks can accumulate and eventually cause high memory usage, slower performance, or system instability.

3. Why is missing upper-bound validation dangerous?

Raw array indexing does not automatically check whether an index is valid. If an index is greater than or equal to the array size, the program may access memory that does not belong to the array. In security-critical systems, this can cause crashes, data corruption, or other serious vulnerabilities.

4. How did eliminating redundant nested loops affect performance?

The deletion routine became simpler by using only one loop to shift the elements. The operation remains $O(n)$, but removing the unnecessary nested loop reduces extra operations and makes the code easier to understand and maintain.

5. Why are peer code audits important in commercial software development?

Peer code audits help find bugs and design problems that the original developer may overlook. They also improve code quality, readability, security, and maintainability before changes are merged into a production codebase.

Reflection

1. Experiential Reflection

The most challenging defect to locate was the memory leak during dynamic array resizing. The code still appeared to work because the elements were copied correctly, but the old memory was never released. Unit testing helped identify the problem because the resize test and memory diagnostics showed that memory was being lost. This was more reliable than simply running the program and guessing where the problem was.

2. Architectural Evaluation

For a system that requires frequent insertions in the middle of a list, I would consider using a linked list. A linked list does not require reallocating the entire buffer when inserting a new element. It can be useful when frequent insertions and deletions are more important than fast random access.

3. Professional Growth

Reviewing another programmer's code showed me that working code is not always good-quality code. I learned to pay more attention to memory management, validation, naming, comments, and algorithm efficiency. For my upcoming projects, I will try to write code that is not only functional but also easier for other developers to understand and maintain.